Algoritmos e implementação

Prof. Dr. Hendrik Macedo

Departamento de Computação

Implementação de algoritmos de IA

Rotina

1. Pseudocódigo fornecido
2. Geração de código equivalente em Python
3. Marcações de correspondência
4. Histórico do *peer programming* com IA*
5. Resultados de execuções
6. Submissão via SIGAA

Pontuação: 3pt / 10 pt (Provas práticas)

Exemplo: A*

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s′ ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s′)
    yield NODE(STATE=s′, PARENT=node, ACTION=action, PATH-COST=cost)
```

Exemplo: A*

Can you generate a **C++** implementation of the **A*** algorithm to solve a **maze**?

Histórico da conversa

Exemplo: A*

```
function BEST-FIRST-SEARCH(problem, f) returns a solut
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an
  reached ← a lookup table, with one entry with key proble
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reache
        reached[s] ← child
        add child to frontier
  return failure

function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
```

```cpp
vector<shared_ptr<Node>> a_star(int n, int m, const vector<vector<int>
    priority_queue<shared_ptr<Node>, vector<shared_ptr<Node>>, Compare
    vector<vector<bool>> visited(n, vector<bool>(m, false));
    auto start = make_shared<Node>(sx, sy, 0, abs(sx - ex) + abs(sy -
    pq.push(start);
    visited[sx][sy] = true; // Mark the start node as visited
    while (!pq.empty()) {
        auto node = pq.top();
        pq.pop();
        if (node->x == ex && node->y == ey) {
            vector<shared_ptr<Node>> path;
            while (node) {
                path.push_back(node);
                node = node->parent;
            }
            reverse(path.begin(), path.end());
            return path;
        }
        auto neighbors = get_neighbors(node, n, m, maze, ex, ey);
        for (auto& neighbor : neighbors) {
            if (!visited[neighbor->x][neighbor->y]) {
                pq.push(neighbor);
                visited[neighbor->x][neighbor->y] = true; // Mark the
            }
        }
    }
    return {};
}
```